AI Assisted Coding
Lab Assignment - 9.5

Name : Shivanand Rama
Hall.t.No. : 2303A51037
Batch No. : 01

Lab Experiment: Documentation Generation -Automatic documentation and code comments

Lab Objectives

1. To understand automatic documentation generation.
2. To generate code comments and docstrings using AI tools.
3. To learn the importance of documentation in software development.

Lab Outcomes

1. Students will be able to generate documentation automatically for code.
2. Students will be able to add clear comments and docstrings to programs.
3. Students will be able to improve code readability and maintainability using documentation.

Problem 1: String Utilities Function Consider the following Python function:

```
def reverse_string(text):
    return text[::-1]
```

Task:

1. Write documentation in:
   - (a) Docstring
   - (b) Inline comments
   - (c) Google-style documentation
2. Compare the three documentation styles.
3. Recommend the most suitable style for a utility-based string library.

Copilot Suggested Code :

task_1.py > ...

```python
# String Utilities Function Consider the following Python function:
# def reverse_string(text):
# return text[::-1]

def reverse_string(text):
    """
    Reverses the given string.

    Args:
        text (str): The string to be reversed.

    Returns:
        str: The reversed string.
    """
    # Use slicing to reverse the string
    return text[::-1]
# Comparison of documentation styles:
# 1. Docstring: This style is concise and provides a clear description of the function, its parameters, and return value.
# It is commonly used in Python and can be easily accessed using the help() function.

# 2. Inline comments: This style is useful for explaining specific lines of code or logic within the function.
# However, it may not provide a comprehensive overview of the function's purpose and usage.

# 3. Google-style documentation: This style is more detailed and structured, providing a clear overview of the function,
# its parameters, return value, and any exceptions that may be raised. It is often used in larger projects and can be easily
# generated into documentation using tools like Sphinx.

# Recommendation: For a utility-based string library, the most suitable documentation style would be the Google-style documentation.
# This style provides a comprehensive overview of the function, making it easier for users to understand its purpose, parameters, and return value.
# Additionally, it can be easily generated into documentation for larger projects, which is beneficial for a library that may have multiple functions
# and users.
```

Commands Used :

```
PS C:\B.Tech\3-2\AI_Assisted_Coding\lab_assignment-9.5> & "C:\Program Files\Python313\python.exe" c:/B.Tech/3-2/AI_Assisted_Coding/lab_assignment-9.5/task_1.py
PS C:\B.Tech\3-2\AI_Assisted_Coding\lab_assignment-9.5> python -m pydoc task_1
Help on module task_1:

NAME
    task_1

DESCRIPTION
    # String Utilities Function Consider the following Python function:
    # def reverse_string(text):
    # return text[::-1]

FUNCTIONS
    reverse_string(text)
        Reverses the given string.

        Args:
            text (str): The string to be reversed.

        Returns:
            str: The reversed string.

FILE
    c:\b.tech\3-2\ai_assisted_coding\lab_assignment-9.5\task_1.py


PS C:\B.Tech\3-2\AI_Assisted_Coding\lab_assignment-9.5> python -m pydoc -w task_1
wrote task_1.html
PS C:\B.Tech\3-2\AI_Assisted_Coding\lab_assignment-9.5> python -m pydoc -p 8080
Server ready at http://localhost:8080/
Server commands: [b]rowser, [q]uit
server> b
server>
```

After typing these commands an html file is generated :

task_1.html > html

```html
<!DOCTYPE html>
<html lang="en">
<head>
<meta charset="utf-8">
<title>Python: module task_1</title>
</head><body>

<table class="heading">
<tr class="heading-text decor">
<td class="title"> <br><strong class="title">task_1</strong></td>
<td class="extra"><a href=".">index</a><br><a href="file:c%3A%5Cb.tech%5C3-2%5Cai_assisted_coding%5Clab_assignment-9.5%5Ctask_1.py">c:\b.tech\3-2\ai_assiste
    <p><span class="code"># String Utilities Function Consider the following Python function:<br>
# def <a href="#-reverse_string">reverse_string</a>(text):<br>
# return text[::-1]</span></p>
<p>
<table class="section">
<tr class="decor functions-decor heading-text">
<td class="section-title" colspan=3> <br><strong class="bigsection">Functions</strong></td></tr>

<tr><td class="decor functions-decor"><span class="code">      </span></td><td> </td>
<td class="singlecolumn"><dl><dt><a name="-reverse_string"><strong>reverse_string</strong></a>(text)</dt><dd><span class="code">Reverses the giver
 <br>
Args:<br>
    text (str): The string to be reversed.<br>
 <br>
Returns:<br>
    str: The reversed string.</span></dd></dl>
</td></tr></table>
</body></html>
```

The output of this html file is :

index
task_1 c:\b.tech\3-2\ai_assisted_coding\lab_assignment-9.5\task_1.py

String Utilities Function Consider the following Python function:
def reverse_string(text):
return text[::-1]

Functions

reverse_string(text)
Reverses the given string.

Args:
text (str): The string to be reversed.

Returns:
str: The reversed string.

With the html file a webpage is also created with the commands we typed earlier :

Python 3.13.7 [tags/v3.13.7:bcee1c3, MSC v.1944 64 bit (AMD64)]
Windows-11

Module Index : Topics : Keywords
Get Search

task_1

index
c:\b.tech\3-2\ai_assisted_coding\lab_assignment-9.5\task_1.py

```
# String Utilities Function Consider the following Python function:
# def reverse_string(text):
# return text[::-1]
```

Functions

reverse_string(text)

```
    Reverses the given string.

    Args:
        text (str): The string to be reversed.

    Returns:
        str: The reversed string.
```

Problem 2: Password Strength Checker Consider the function:

```
def check_strength(password):
        return len(password) >= 8
```

Task:

1. Document the function using docstring, inline comments, and Google style.
2. Compare documentation styles for security-related code.
3. Recommend the most appropriate style.

Copilot Suggested Code :

task_2.py > ...

```python
# Problem 2: Password Strength Checker Consider the function:
#   def check_strength(password):
#       return len(password) ≥ 8

def check_strength(password):
    """
    Check the strength of a given password.

    This function evaluates the strength of a password based on its length.
    A password is considered strong if it has at least 8 characters.

    Args:
        password (str): The password to be evaluated.
    Returns:
        bool: True if the password is strong, False otherwise.
    Example:
        >>> check_strength("password123")
        True
        >>> check_strength("pass")
        False
    """
    # Check if the length of the password is greater than or equal to 8
    return len(password) ≥ 8
# Comparison of documentation styles for security-related code:
# 1. Docstring: Provides a clear and concise description of the function, its parameters, return value, and examples.
# It is easily accessible and can be used by tools like Sphinx for generating documentation.

# 2. Inline comments: Useful for explaining specific lines of code or logic, but can become cluttered if overused.
# It may not provide a comprehensive overview of the function's purpose and usage.

# 3. Google style: A specific format for docstrings that includes sections for Args, Returns, and Examples.
# It is widely adopted and promotes consistency in documentation.

# Recommendation:
# The most appropriate style for documenting security-related code is the Google style.
# It provides a structured and standardized format that enhances readability and clarity.
# The use of sections for arguments, return values, and examples helps developers understand the function's purpose and how to use it effectively,
# which is crucial for security-related code.
```

Commands Used :

```
PS C:\B.Tech\3-2\AI_Assisted_Coding\lab_assignment-9.5> & "C:\Program Files\Python313\python.exe" c:/B.Tech/3-2/AI_Assisted_Coding/lab_assignment-9.5/task_2.py
PS C:\B.Tech\3-2\AI_Assisted_Coding\lab_assignment-9.5> python -m pydoc task_2
Help on module task_2:

NAME
    task_2

DESCRIPTION
    # Problem 2: Password Strength Checker Consider the function:
    #   def check_strength(password):
    #       return len(password) >= 8

FUNCTIONS
    check_strength(password)
        Check the strength of a given password.

        This function evaluates the strength of a password based on its length.
        A password is considered strong if it has at least 8 characters.

        Args:
            password (str): The password to be evaluated.
        Returns:
            bool: True if the password is strong, False otherwise.
        Example:
            >>> check_strength("password123")
            True
            >>> check_strength("pass")
            False

FILE
    c:\b.tech\3-2\ai_assisted_coding\lab_assignment-9.5\task_2.py


PS C:\B.Tech\3-2\AI_Assisted_Coding\lab_assignment-9.5> python -m pydoc -w task_2
wrote task_2.html
PS C:\B.Tech\3-2\AI_Assisted_Coding\lab_assignment-9.5> python -m pydoc -p 8080
Server ready at http://localhost:8080/
Server commands: [b]rowser, [q]uit
server> b
server>
```

After typing these commands an html file is generated :

task_2.html > html

```
<!DOCTYPE html>
<html lang="en">
<head>
<meta charset="utf-8">
<title>Python: module task_2</title>
</head><body>

<table class="heading">
<tr class="heading-text decor">
<td class="title"> <br><strong class="title">task_2</strong></td>
<td class="extra"><a href=".">index</a><br><a href="file:c%3A%5Cb.tech%5C3-2%5Cai_assisted_coding%5Clab_assignment-9.5%5Ctask_2.py">c:\b.tech\3-2\ai_assiste
    <p><span class="code"># Problem 2: Password Strength Checker Consider the function:<br>
#   def <a href="#-check_strength">check_strength</a>(password):<br>
#       return len(password) &gt;= 8</span></p>
<p>
<table class="section">
<tr class="decor functions-decor heading-text">
<td class="section-title" colspan=3> <br><strong class="bigsection">Functions</strong></td></tr>

<tr><td class="decor functions-decor"><span class="code">      </span></td><td> </td>
<td class="singlecolumn"><dl><dt><a name="-check_strength"><strong>check_strength</strong></a>(password)</dt><dd><span class="code">Check the stre
 <br>
This function evaluates the strength of a password based on its length.<br>
A password is considered strong if it has at least 8 characters.<br>
 <br>
Args:<br>
        password (str): The password to be evaluated.<br>
Returns:<br>
        bool: True if the password is strong, False otherwise.<br>
Example:<br>
        &gt;&gt;&gt; <a href="#-check_strength">check_strength</a>("password123")<br>
        True<br>
        &gt;&gt;&gt; <a href="#-check_strength">check_strength</a>("pass")<br>
        False</span></dd></dl>
</td></tr></table>
</body></html>
```

The output of this html file is :

index
task_2 c:\b.tech\3-2\ai_assisted_coding\lab_assignment-9.5\task_2.py

```
# Problem 2: Password Strength Checker Consider the function:
#   def check_strength(password):
#       return len(password) >= 8
```

Functions

check_strength(password)
Check the strength of a given password.

This function evaluates the strength of a password based on its length.
A password is considered strong if it has at least 8 characters.

Args:
  password (str): The password to be evaluated.
Returns:
  bool: True if the password is strong, False otherwise.
Example:
  >>> check_strength("password123")
  True
  >>> check_strength("pass")
  False

With the html file a webpage is also created with the commands we typed earlier :

Python 3.13.7 [tags/v3.13.7:bcee1c3, MSC v.1944 64 bit (AMD64)]
Windows-11

Module Index : Topics : Keywords
Get Search

task_2

index
c:\b.tech\3-2\ai_assisted_coding\lab_assignment-9.5\task_2.py

```
# Problem 2: Password Strength Checker Consider the function:
#   def check_strength(password):
#       return len(password) >= 8
```

Functions

check_strength(password)
Check the strength of a given password.

This function evaluates the strength of a password based on its length.
A password is considered strong if it has at least 8 characters.

Args:
  password (str): The password to be evaluated.
Returns:
  bool: True if the password is strong, False otherwise.
Example:
  >>> check_strength("password123")
  True
  >>> check_strength("pass")
  False

Problem 3: Math Utilities Module

Task:

1. Create a module math_utils.py with functions:
    - o square(n)
    - o cube(n)
    - o factorial(n)
2. Generate docstrings automatically using AI tools.
3. Export documentation as an HTML file.

Copilot Suggested Code :

task_3.py > ...

```python
# Create a module math_utils.py with functions:
# o square(n)
# o cube(n)
# o factorial(n)

# math_utils.py
def square(n):
    """
    Calculate the square of a number.

    Parameters:
    n (int or float): The number to be squared.

    Returns:
    int or float: The square of the input number.
    """
    return n * n
def cube(n):
    """Calculate the cube of a number.
    Parameters:
    n (int or float): The number to be cubed.
    Returns:
    int or float: The cube of the input number.
    """
    return n * n * n
def factorial(n):
    """Calculate the factorial of a number.
    Parameters:
    n (int): The number to calculate the factorial for. Must be a non-negative integer.
    Returns:
    int: The factorial of the input number.
    Raises:
    ValueError: If n is negative or not an integer.
    """
    if n < 0 or not isinstance(n, int):
        raise ValueError("Input must be a non-negative integer.")
    if n == 0 or n == 1:
        return 1
    result = 1
    for i in range(2, n + 1):
        result *= i
    return result

# To export documentation as an HTML file, you can use the pydoc module in Python.
# Run the following command in the terminal to generate the HTML documentation:
# pydoc -w math_utils
# This will create a file named math_utils.html in the current directory, which contains the documentation for the math_utils module.
```

Commands Used :

```
PS C:\B.Tech\3-2\AI_Assisted_Coding\lab_assignment-9.5> python -m pydoc task_3
Help on module task_3:

NAME
    task_3

DESCRIPTION
    # Create a module math_utils.py with functions:
    # o square(n)
    # o cube(n)
    # o factorial(n)

FUNCTIONS
    cube(n)
        Calculate the cube of a number.
        Parameters:
        n (int or float): The number to be cubed.
        Returns:
        int or float: The cube of the input number.

    factorial(n)
        Calculate the factorial of a number.
        Parameters:
        n (int): The number to calculate the factorial for. Must be a non-negative integer.
        Returns:
        int: The factorial of the input number.
        Raises:
        ValueError: If n is negative or not an integer.

    square(n)
        Calculate the square of a number.

        Parameters:
        n (int or float): The number to be squared.

        Returns:
        int or float: The square of the input number.

FILE
    c:\b.tech\3-2\ai_assisted_coding\lab_assignment-9.5\task_3.py


PS C:\B.Tech\3-2\AI_Assisted_Coding\lab_assignment-9.5> python -m pydoc -w task_3
wrote task_3.html
PS C:\B.Tech\3-2\AI_Assisted_Coding\lab_assignment-9.5> python -m pydoc -p 8080
Server ready at http://localhost:8080/
Server commands: [b]rowser, [q]uit
server> b
server>
```

After typing these commands an html file is generated :

task_3.html > html

```html
<!DOCTYPE html>
<html lang="en">
<head>
<meta charset="utf-8">
<title>Python: module task_3</title>
</head><body>

<table class="heading">
<tr class="heading-text decor">
<td class="title"> <br><strong class="title">task_3</strong></td>
<td class="extra"><a href=".">index</a><br><a href="file:c%3A%5Cb.tech%5C3-2%5Cai_assisted_coding%5Clab_assignment-9.5%5Ctask_3.py">c:\b.tech\3-2\ai_assiste
    <p><span class="code"># Create a module math_utils.py with functions:<br>
# o <a href="#-square">square</a>(n)<br>
# o <a href="#-cube">cube</a>(n)<br>
# o <a href="#-factorial">factorial</a>(n)</span></p>
<p>
<table class="section">
<tr class="decor functions-decor heading-text">
<td class="section-title" colspan=3> <br><strong class="bigsection">Functions</strong></td></tr>

<tr><td class="decor functions-decor"><span class="code">      </span></td><td> </td>
<td class="singlecolumn"><dl><dt><a name="-cube"><strong>cube</strong></a>(n)</dt><dd><span class="code">Calculate the cube of a nu
Parameters:<br>
n (int or float): The number to be cubed.<br>
Returns:<br>
int or float: The cube of the input number.</span></dd></dl>
 <dl><dt><a name="-factorial"><strong>factorial</strong></a>(n)</dt><dd><span class="code">Calculate the factorial of a number.<br>
Parameters:<br>
n (int): The number to calculate the factorial for. Must be a non-negative integer.<br>
Returns:<br>
int: The factorial of the input number.<br>
Raises:<br>
ValueError: If n is negative or not an integer.</span></dd></dl>
 <dl><dt><a name="-square"><strong>square</strong></a>(n)</dt><dd><span class="code">Calculate the square of a number.<br>
 <br>
Parameters:<br>
n (int or float): The number to be squared.<br>
 <br>
Returns:<br>
int or float: The square of the input number.</span></dd></dl>
</td></tr></table>
</body></html>
```

The output of this html file is :

index
task_3 c:\b.tech\3-2\ai_assisted_coding\lab_assignment-9.5\task_3.py

Create a module math_utils.py with functions:
o square(n)
o cube(n)
o factorial(n)

Functions

cube(n)
Calculate the cube of a number.
Parameters:
n (int or float): The number to be cubed.
Returns:
int or float: The cube of the input number.

factorial(n)
Calculate the factorial of a number.
Parameters:
n (int): The number to calculate the factorial for. Must be a non-negative integer.
Returns:
int: The factorial of the input number.
Raises:
ValueError: If n is negative or not an integer.

square(n)
Calculate the square of a number.

Parameters:
n (int or float): The number to be squared.

Returns:
int or float: The square of the input number.

With the html file a webpage is also created with the commands we typed earlier :

Problem 4: Attendance Management Module

Task:

1. Create a module attendance.py with functions:
    - o mark_present(student)
    - o mark_absent(student)
    - o get_attendance(student)
2. Add proper docstrings.
3. Generate and view documentation in terminal and browse

Copilot Suggested Code :

task_4.py > ...

```python
# Create a module attendance.py with functions:
# o mark_present(student)
# o mark_absent(student)
# o get_attendance(student)

# attendance.py
class Attendance:
    def __init__(self):
        """Initialize the attendance record."""
        self.attendance_record = {}

    def mark_present(self, student):
        """Mark a student as present.
        Parameters:
        student (str): The name of the student to be marked as present.
        """
        self.attendance_record[student] = 'Present'

    def mark_absent(self, student):
        """Mark a student as absent.
        Parameters:
        student (str): The name of the student to be marked as absent.
        """
        self.attendance_record[student] = 'Absent'

    def get_attendance(self, student):
        """Get the attendance status of a student.
        Parameters:
        student (str): The name of the student whose attendance status is to be retrieved.
        Returns:
        str: The attendance status of the student ('Present', 'Absent', or 'Not Recorded').
        """
        return self.attendance_record.get(student, 'Not Recorded')
# To export documentation as an HTML file, you can use the pydoc module in Python.
# Run the following command in the terminal to generate the HTML documentation:
# pydoc -w attendance
# This will create a file named attendance.html in the current directory, which contains the documentation for the attendance module.
```

Commands Used :

```
PS C:\B.Tech\3-2\AI_Assisted_Coding\lab_assignment-9.5> & "C:\Program Files\Python313\python.exe" c:/B.Tech/3-2/AI_Assisted_Coding/lab_assignment-9.5/task_4.py
PS C:\B.Tech\3-2\AI_Assisted_Coding\lab_assignment-9.5> python -m pydoc task_4
Help on module task_4:

NAME
    task_4

DESCRIPTION
    # Create a module attendance.py with functions:
    # o mark_present(student)
    # o mark_absent(student)
    # o get_attendance(student)

CLASSES
    builtins.object
        Attendance

    class Attendance(builtins.object)
     |  # attendance.py
     |
     |  Methods defined here:
     |
     |  __init__(self)
     |      Initialize the attendance record.
     |
     |  get_attendance(self, student)
     |      Get the attendance status of a student.
     |      Parameters:
     |      student (str): The name of the student whose attendance status is to be retrieved.
     |      Returns:
     |      str: The attendance status of the student ('Present', 'Absent', or 'Not Recorded').
     |
     |  mark_absent(self, student)
     |      Mark a student as absent.
     |      Parameters:
     |      student (str): The name of the student to be marked as absent.
     |
     |  mark_present(self, student)
     |      Mark a student as present.
     |      Parameters:
     |      student (str): The name of the student to be marked as present.
     |
     |  ----------------------------------------------------------------------
     |  Data descriptors defined here:
     |
     |  __dict__
     |      dictionary for instance variables
     |
     |  __weakref__
     |      list of weak references to the object

FILE
    c:\b.tech\3-2\ai_assisted_coding\lab_assignment-9.5\task_4.py


PS C:\B.Tech\3-2\AI_Assisted_Coding\lab_assignment-9.5> python -m pydoc -w task_4
wrote task_4.html
PS C:\B.Tech\3-2\AI_Assisted_Coding\lab_assignment-9.5> python -m pydoc -p 8080
Server ready at http://localhost:8080/
Server commands: [b]rowser, [q]uit
server> b
server>
```

After typing these commands an html file is generated :

task_4.html > html

```html
<!DOCTYPE html>
<html lang="en">
<head>
<meta charset="utf-8">
<title>Python: module task_4</title>
</head><body>

<table class="heading">
<tr class="heading-text decor">
<td class="title"> <br><strong class="title">task_4</strong></td>
<td class="extra"><a href=".">index</a><br><a href="file:c%3A%5Cb.tech%5C3-2%5Cai_assisted_coding%5Clab_assignment-9.5%5Ctask_4.py">c:\b.tech\3-2\ai_assisted_coding\lab_assignment-9.5\task_4.py</a></td></tr></table>
    <p><span class="code"># Create a module attendance.py with functions:<br>
# o mark_present(student)<br>
# o mark_absent(student)<br>
# o get_attendance(student)</span></p>
<p>
<table class="section">
<tr class="decor index-decor heading-text">
<td class="section-title" colspan=3> <br><strong class="bigsection">Classes</strong></td></tr>

<tr><td class="decor index-decor"><span class="code">      </span></td><td> </td>
<td class="singlecolumn"><dl>
<dt class="heading-text"><a href="builtins.html#object">builtins.object</a>
</dt><dd>
<dl>
<dt class="heading-text"><a href="task_4.html#Attendance">Attendance</a>
</dt></dl>
</dd>
</dl>
 <p>
<table class="section">
<tr class="decor title-decor heading-text">
<td class="section-title" colspan=3> <br><a name="Attendance">class <strong>Attendance</strong></a>(<a href="builtins.html#object">builtins.object</a>)</td></tr>

<tr><td class="decor title-decor" rowspan=2><span class="code">   </span></td>
<td class="decor title-decor" colspan=2><span class="code"># attendance.py<br> </span></td></tr>
<tr><td> </td>
<td class="singlecolumn">Methods defined here:<br>
<dl><dt><a name="Attendance-__init__"><strong>__init__</strong></a>(self)</dt><dd><span class="code">Initialize the attendance record.</span></dd></dl>

<dl><dt><a name="Attendance-get_attendance"><strong>get_attendance</strong></a>(self, student)</dt><dd><span class="code">Get the attendance status of a student.<br>
Parameters:<br>
student (str): The name of the student whose attendance status is to be retrieved.<br>
Returns:<br>
str: The attendance status of the student ('Present', 'Absent', or 'Not Recorded').</span></dd></dl>

<dl><dt><a name="Attendance-mark_absent"><strong>mark_absent</strong></a>(self, student)</dt><dd><span class="code">Mark a student as absent.<br>
Parameters:<br>
student (str): The name of the student to be marked as absent.</span></dd></dl>

<dl><dt><a name="Attendance-mark_present"><strong>mark_present</strong></a>(self, student)</dt><dd><span class="code">Mark a student as present.<br>
Parameters:<br>
student (str): The name of the student to be marked as present.</span></dd></dl>

<hr>
Data descriptors defined here:<br>
<dl><dt><strong>__dict__</strong></dt>
<dd><span class="code">dictionary for instance variables</span></dd>
</dl>
<dl><dt><strong>__weakref__</strong></dt>
<dd><span class="code">list of weak references to the object</span></dd>
</dl>
</td></tr></table></td></tr></table>
</body></html>
```

The output of this html file is :

index
task_4 c:\b.tech\3-2\ai_assisted_coding\lab_assignment-9.5\task_4.py

```
# Create a module attendance.py with functions:
# o mark_present(student)
# o mark_absent(student)
# o get_attendance(student)
```

Classes

builtins.object
Attendance

class **Attendance**(builtins.object)
attendance.py

Methods defined here:

__init__(self)
Initialize the attendance record.

get_attendance(self, student)
Get the attendance status of a student.
Parameters:
student (str): The name of the student whose attendance status is to be retrieved.
Returns:
str: The attendance status of the student ('Present', 'Absent', or 'Not Recorded').

mark_absent(self, student)
Mark a student as absent.
Parameters:
student (str): The name of the student to be marked as absent.

mark_present(self, student)
Mark a student as present.
Parameters:
student (str): The name of the student to be marked as present.

Data descriptors defined here:

__dict__
dictionary for instance variables

__weakref__
list of weak references to the object

With the html file a webpage is also created with the commands we typed earlier :

Python 3.13.7 [tags/v3.13.7:bcee1c3, MSC v.1944 64 bit (AMD64)]
Windows-11

Module Index : Topics : Keywords
Get Search

task_4

index
c:\b.tech\3-2\ai_assisted_coding\lab_assignment-9.5\task_4.py

```
# Create a module attendance.py with functions:
# o mark_present(student)
# o mark_absent(student)
# o get_attendance(student)
```

Classes

builtins.object
Attendance

class **Attendance**(builtins.object)

```
# attendance.py
```

Methods defined here:

__init__(self)
Initialize the attendance record.

get_attendance(self, student)
Get the attendance status of a student.
Parameters:
student (str): The name of the student whose attendance status is to be retrieved.
Returns:
str: The attendance status of the student ('Present', 'Absent', or 'Not Recorded').

mark_absent(self, student)
Mark a student as absent.
Parameters:
student (str): The name of the student to be marked as absent.

mark_present(self, student)
Mark a student as present.
Parameters:
student (str): The name of the student to be marked as present.

Data descriptors defined here:

__dict__
dictionary for instance variables

__weakref__
list of weak references to the object

Problem 5: File Handling Function

Consider the function:

```
def read_file(filename):
    with open(filename, 'r') as f:
        return f.read()
```

Task:

1. Write documentation using all three formats.
2. Identify which style best explains exception handling.
3. Justify your recommendation.

Copilot Suggested Code :

task_5.py > ...

```python
# File Handling Function
# Consider the function:
#   def read_file(filename):
#       with open(filename, 'r') as f:
#           return f.read()

def read_file(filename):
    """
    Reads the content of a file and returns it as a string.

    Parameters:
    filename (str): The name of the file to be read.

    Returns:
    str: The content of the file.

    Raises:
    FileNotFoundError: If the specified file does not exist.
    IOError: If an I/O error occurs while reading the file.
    """
    try:
        with open(filename, 'r') as f:
            return f.read()
    except FileNotFoundError:
        print(f"Error: The file '{filename}' was not found.")
        raise
    except IOError as e:
        print(f"An I/O error occurred: {e}")
        raise
# The style that best explains exception handling is the Google style. This is because it includes a clear "Raises" section that explicitly lists
# the exceptions that can be raised by the function, along with descriptions of the circumstances under which they may occur.
# This makes it easier for developers to understand the potential errors and how to handle them effectively.

# The Google style is recommended for documenting exception handling because it provides a structured and comprehensive way to communicate
# the possible exceptions, which can help developers write more robust and error-resistant code. By clearly outlining the exceptions,
# developers can anticipate and manage errors more effectively, leading to better user experience and improved code quality.
```

Commands Used :

```
PS C:\B.Tech\3-2\AI_Assisted_Coding\lab_assignment-9.5> & "C:\Program Files\Python313\python.exe" c:/B.Tech/3-2/AI_Assisted_Coding/lab_assignment-9.5/task_5.py
PS C:\B.Tech\3-2\AI_Assisted_Coding\lab_assignment-9.5> python -m pydoc task_5
Help on module task_5:

NAME
    task_5

DESCRIPTION
    # File Handling Function
    # Consider the function:
    #   def read_file(filename):
    #       with open(filename, 'r') as f:
    #           return f.read()

FUNCTIONS
    read_file(filename)
        Reads the content of a file and returns it as a string.

        Parameters:
        filename (str): The name of the file to be read.

        Returns:
        str: The content of the file.

        Raises:
        FileNotFoundError: If the specified file does not exist.
        IOError: If an I/O error occurs while reading the file.

FILE
    c:\b.tech\3-2\ai_assisted_coding\lab_assignment-9.5\task_5.py


PS C:\B.Tech\3-2\AI_Assisted_Coding\lab_assignment-9.5> python -m pydoc -w task_5
wrote task_5.html
PS C:\B.Tech\3-2\AI_Assisted_Coding\lab_assignment-9.5> python -m pydoc -p 8080
Server ready at http://localhost:8080/
Server commands: [b]rowser, [q]uit
server> b
server>
```

After typing these commands an html file is generated :

task_5.html > html

```html
<!DOCTYPE html>
<html lang="en">
<head>
<meta charset="utf-8">
<title>Python: module task_5</title>
</head><body>

<table class="heading">
<tr class="heading-text decor">
<td class="title"> <br><strong class="title">task_5</strong></td>
<td class="extra"><a href=".">index</a><br><a href="file:c%3A%5Cb.tech%5C3-2%5Cai_assisted_coding%5Clab_assignment-9.5%5Ctask_5.py">c:\b.tech\3-2\ai_assiste
    <p><span class="code"># File Handling Function<br>
# Consider the function:<br>
#   def <a href="#-read_file">read_file</a>(filename):<br>
#       with open(filename, 'r') as f:<br>
#            return f.read()</span></p>
<p>
<table class="section">
<tr class="decor functions-decor heading-text">
<td class="section-title" colspan=3> <br><strong class="bigsection">Functions</strong></td></tr>

<tr><td class="decor functions-decor"><span class="code">       </span></td><td> </td>
<td class="singlecolumn"><dl><dt><a name="-read_file"><strong>read_file</strong></a>(filename)</dt><dd><span class="code">Reads the content c
 <br>
Parameters:<br>
filename (str): The name of the file to be read.<br>
 <br>
Returns:<br>
str: The content of the file.<br>
 <br>
Raises:<br>
FileNotFoundError: If the specified file does not exist.<br>
IOError: If an I/O error occurs while reading the file.</span></dd></dl>
</td></tr></table>
</body></html>
```

The output of this html file is :

index
task_5 c:\b.tech\3-2\ai_assisted_coding\lab_assignment-9.5\task_5.py

```
# File Handling Function
# Consider the function:
#   def read_file(filename):
#       with open(filename, 'r') as f:
#           return f.read()
```

Functions

read_file(filename)
Reads the content of a file and returns it as a string.

Parameters:
filename (str): The name of the file to be read.

Returns:
str: The content of the file.

Raises:
FileNotFoundError: If the specified file does not exist.
IOError: If an I/O error occurs while reading the file.

With the html file a webpage is also created with the commands we typed earlier :

Python 3.13.7 [tags/v3.13.7:bcee1c3, MSC v.1944 64 bit (AMD64)]
Windows-11

Module Index : Topics : Keywords
Get Search

task_5

index
c:\b.tech\3-2\ai_assisted_coding\lab_assignment-9.5\task_5.py

```
# File Handling Function
# Consider the function:
#   def read_file(filename):
#       with open(filename, 'r') as f:
#           return f.read()
```

Functions

read_file(filename)

```
Reads the content of a file and returns it as a string.

Parameters:
filename (str): The name of the file to be read.

Returns:
str: The content of the file.

Raises:
FileNotFoundError: If the specified file does not exist.
IOError: If an I/O error occurs while reading the file.
```